INTERNSHIP PROJECT REPORT

Pearls AQI Predictor

A Serverless Air Quality Index Forecasting, Monitoring and Explainable ML System

Internship Duration: 8 Weeks
Submitted By: Areeba Mandavia

Organization: 10 Pearls

Department: Data Science

Internship Period: July 13 – September 4, 2026

1. Executive Summary

Air pollution is an important environmental issue in large cities. The Air Quality Index (AQI) provides a simplified representation of air pollution levels and helps users understand the severity of air pollution. However, monitoring only the current AQI does not provide information about how air quality may change in the near future.

During my eight-week internship, I developed **Pearls AQI Predictor**, an end-to-end machine learning system for monitoring and forecasting air quality in Karachi, Pakistan.

The system collects hourly air quality and weather information from public APIs. Historical data is processed, cleaned and combined before undergoing extensive feature engineering. Time-based features, lag features, rolling statistics and pollutant-change features are generated to capture temporal patterns in air quality.

Multiple machine learning approaches were evaluated, including a persistence baseline, Ridge Regression, Random Forest and XGBoost. The final system uses **XGBoost for 72-hour AQI forecasting**, allowing the system to generate a three-day AQI forecast.

The project also incorporates **Hopsworks** as part of its machine learning infrastructure. Hopsworks Feature Store is used to organize and manage engineered features, while the Hopsworks project and model-management capabilities provide a structured environment for machine learning artifacts and model lifecycle management.

SHAP (SHapley Additive exPlanations) was integrated to provide model explainability. This allows the system to identify which environmental, pollution and temporal features contribute to AQI predictions.

A **Streamlit dashboard** was developed as the user-facing component of the system. The dashboard is designed around three major functions:

- Displaying the **current AQI reading**
- Displaying a **3-day / 72-hour AQI forecast**
- Providing **SHAP-based explainability** for model predictions

GitHub Actions was used to automate the data collection and model-training workflows.

The final architecture combines data engineering, machine learning, feature management, explainability, workflow automation and web deployment into a single end-to-end environmental forecasting system.

---

2. Introduction

Air pollution is a major environmental challenge in densely populated urban areas. Karachi experiences variations in air quality due to traffic, industrial activity, weather conditions, atmospheric pressure, wind patterns and other environmental factors.

The Air Quality Index provides a numerical representation of air pollution and allows air quality conditions to be classified into categories such as Good, Moderate and Unhealthy.

Traditional air-quality monitoring systems primarily provide current or historical measurements. Forecasting introduces an additional capability by allowing users to anticipate potential changes in air quality.

The objective of this project was to develop a complete machine learning pipeline capable of collecting environmental data, engineering predictive features, training forecasting models, explaining predictions and presenting results through an interactive dashboard.

The system was designed to perform the following tasks:

- Collect air quality data.
- Collect weather data.
- Store and process historical observations.
- Perform exploratory data analysis.
- Generate time-series machine learning features.
- Store engineered features using a Feature Store.
- Train and evaluate multiple machine learning models.
- Forecast AQI for the next 72 hours.
- Explain model predictions using SHAP.
- Register and manage model information.
- Automate data and model pipelines.
- Display results through an interactive dashboard.
- Deploy the dashboard online.

---

3. Project Objectives

The main objectives of the project were:

1. Develop an automated AQI data collection system.
2. Collect historical air quality and weather data for Karachi.
3. Prepare and validate the collected datasets.
4. Perform exploratory data analysis.
5. Engineer temporal and statistical features.

6. Implement a Feature Store using Hopsworks.
7. Develop and compare multiple machine learning models.
8. Develop a 72-hour AQI forecasting model.
9. Evaluate forecasting performance using MAE, RMSE and R^2.
10. Implement SHAP-based model explainability.
11. Implement model versioning and model management.
12. Automate data and model pipelines using GitHub Actions.
13. Develop an interactive Streamlit dashboard.
14. Display current AQI readings.
15. Display a three-day AQI forecast.
16. Display explainability information for model predictions.
17. Deploy the final machine learning application.

---

4. Project Scope

The project focuses on air quality forecasting for **Karachi, Pakistan**.

The system uses hourly observations containing air pollution and weather variables.

The primary environmental variables include:

- PM2.5
- PM10
- Carbon Monoxide (CO)
- Nitrogen Dioxide (NO2)
- Sulfur Dioxide (SO2)
- Ozone (O3)
- AQI
- Temperature
- Humidity
- Atmospheric pressure
- Wind speed

- Wind direction
- Precipitation

The forecasting component predicts AQI for up to **72 hours into the future**, corresponding to a three-day forecast.

The project also includes:

- Exploratory data analysis
- Feature engineering
- Hopsworks Feature Store
- Machine learning model training
- Model management
- SHAP explainability
- GitHub Actions automation
- Streamlit dashboard
- Cloud deployment

---

5. Technologies Used

5.1 Programming Language

Python was used as the primary programming language.

Python was selected because of its extensive ecosystem for:

- Data engineering
- Machine learning
- Time-series processing
- API integration
- Data visualization
- Model explainability
- Web application development

5.2 Machine Learning and Data Libraries

The major libraries used in the project include:

- Pandas
- NumPy
- Scikit-learn
- XGBoost
- Joblib
- SHAP
- Matplotlib
- Seaborn
- PyArrow
- Requests

5.3 Data Source

The project uses the **Open-Meteo API** to collect air quality and weather information.

The API provides hourly environmental measurements that are used for historical analysis and machine learning.

5.4 Feature Store and MLOps

Hopsworks was incorporated into the project for Feature Store functionality and machine learning infrastructure.

Hopsworks provides a centralized environment for managing machine learning features and supporting model-development workflows.

The Hopsworks integration allows engineered features to be organized independently from the model-training code.

5.5 Version Control

Git and GitHub were used for:

- Source code management
- Version control

- Repository management
- Pipeline automation
- Deployment

5.6 Automation

GitHub Actions was used to automate recurring machine learning workflows.

Two primary workflows were developed:

- Hourly AQI Data Pipeline
- Daily Model Training Pipeline

5.7 Dashboard

Streamlit was used to develop the web-based dashboard.

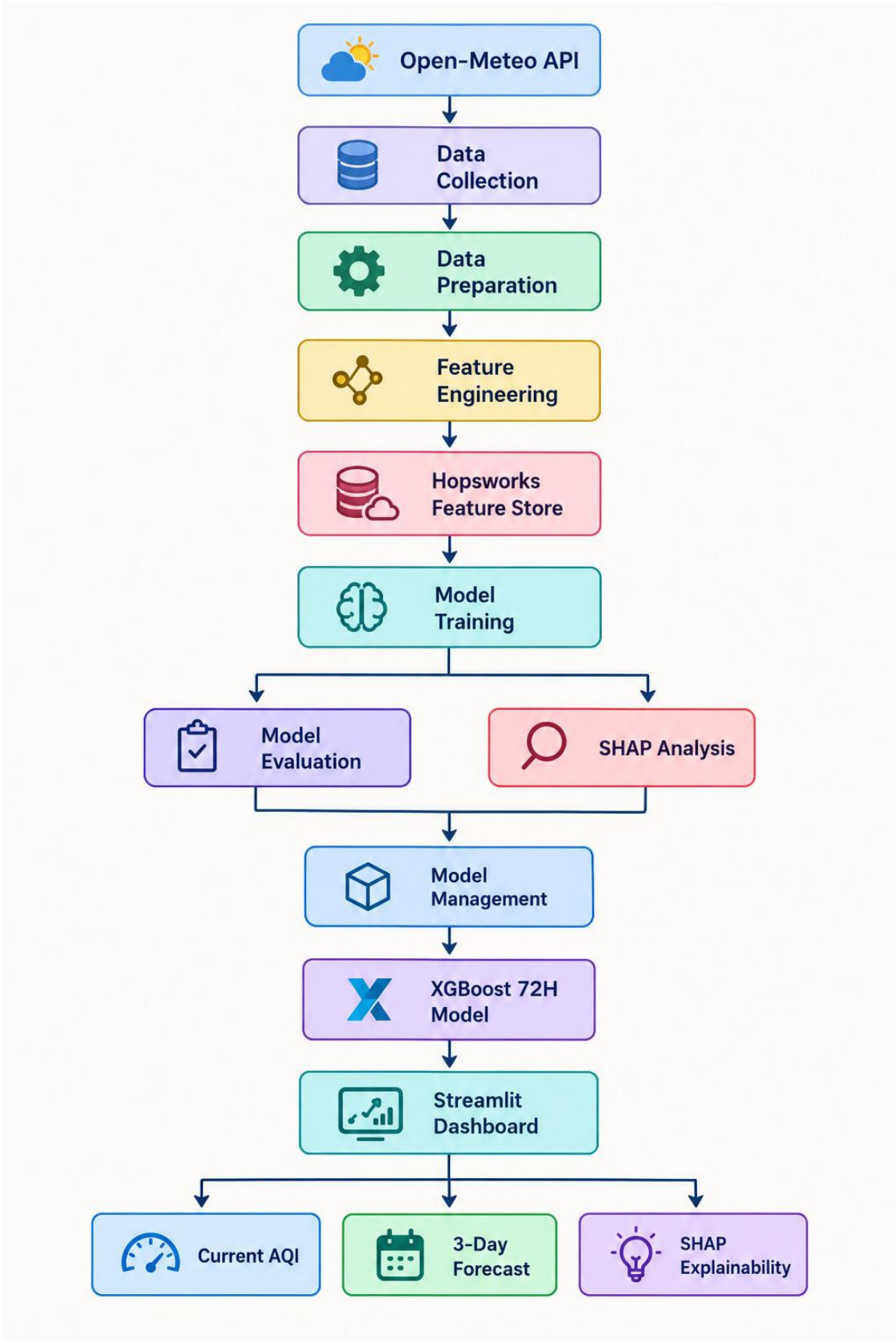
The dashboard provides:

- Current AQI
- Three-day AQI forecast
- Forecast visualization
- SHAP explainability
- Model information

---

6. System Architecture

The overall architecture follows an end-to-end machine learning workflow:


Open-Meteo API
Data Collection
Data Preparation
Feature Engineering
Hopsworks Feature Store
Model Training
Model Evaluation
SHAP Analysis
Model Management
XGBoost 72H Model
Streamlit Dashboard
Current AQI
3-Day Forecast
SHAP Explainability

This architecture separates data collection, feature engineering, feature management, model training, explainability and visualization.

---

7. Data Collection

The first stage of the project was collecting air quality and weather data.

The data collection process connects to the Open-Meteo APIs and retrieves hourly observations for Karachi.

The geographical coordinates used were:

- Latitude: 24.8607
- Longitude: 67.0011

The collected variables include pollutant concentrations, AQI and meteorological variables.

The collection process was implemented in Python and organized as a reusable pipeline.

---

8. Historical Data Backfill

Historical data was required for machine learning model development.

A historical backfill process was developed to collect air quality and weather information from January 2025 through August 2026.

The historical datasets were collected at an hourly frequency.

The resulting datasets were then combined using the timestamp.

The final historical dataset contained:

14,592 hourly records.

This provided sufficient observations for time-series feature engineering and model evaluation.

---

9. Data Preparation

The air quality and weather datasets were merged using the hourly timestamp.

The resulting prepared dataset contains 14 primary variables.

Variable	Description

timestamp	Date and time of observation
pm10	PM10 concentration
pm25	PM2.5 concentration
co	Carbon monoxide
no2	Nitrogen dioxide
so2	Sulfur dioxide
o3	Ozone
aqi	Air Quality Index
temperature	Temperature
humidity	Relative humidity
pressure	Atmospheric pressure
wind_speed	Wind speed
wind_direction	Wind direction
precipitation	Precipitation

The prepared dataset was stored as:

data/processed/karachi_complete.csv

---

10. Exploratory Data Analysis

Exploratory Data Analysis was performed before model training.

The objectives of EDA were to understand:

- Data distributions
- Missing values
- Duplicate timestamps
- AQI categories
- Temporal patterns
- Pollutant behavior
- Relationships between environmental variables

The final merged dataset contained:

14,592 rows and 14 columns.

The dataset was checked for missing values and duplicate timestamps.

The EDA results were saved for further analysis.

---

11. AQI Distribution

The AQI values were categorized according to their severity.

The distribution was:

AQI Category	Records	Percentage
Moderate	11,434	78.36%
Unhealthy for Sensitive Groups	2,825	19.36%
Unhealthy	305	2.09%
Good	28	0.19%

Most observations belonged to the Moderate category.

However, the dataset also contained observations in more severe categories, allowing the model to learn from different air-quality conditions.

The EDA output was stored in:

data/processed/karachi_eda.csv

---

12. Feature Engineering

Feature engineering was a major component of the project.

AQI is a time-dependent environmental variable. Current pollution levels, historical pollution levels, weather conditions and recurring temporal patterns can all influence future AQI.

Therefore, a large set of time-series features was generated.

The final feature dataset contained:

237 columns

The main feature groups included:

- Time features
- Cyclical time features

- AQI lag features
- Pollutant lag features
- Rolling statistics
- AQI change features
- Pollutant change features
- Future AQI targets

---

13. Time Features

The following temporal features were generated:

- Hour
- Day of week
- Day of month
- Month
- Day of year
- Weekend indicator

These features allow the model to identify recurring patterns.

For example, air quality can behave differently during weekdays and weekends.

---

14. Cyclical Time Features

Time variables are cyclical.

For example, 23:00 and 00:00 are consecutive hours, even though their numerical values are far apart.

To represent this relationship, sine and cosine transformations were applied.

The generated features included:

- hour_sin
- hour_cos
- dow_sin
- dow_cos

- doy_sin
- doy_cos

These features help machine learning models understand periodic patterns.

---

15. AQI Lag Features

Historical AQI values were used as predictive features.

The following lag periods were included:

- 1 hour
- 2 hours
- 3 hours
- 6 hours
- 12 hours
- 18 hours
- 24 hours
- 48 hours
- 72 hours
- 168 hours

The 168-hour lag represents one week of historical behavior.

---

16. Pollutant Lag Features

Historical pollutant values were also incorporated.

Lag features were generated for:

- PM2.5
- PM10
- NO2
- SO2
- O3

- CO

Lag periods included:

- 1 hour
- 3 hours
- 6 hours
- 12 hours
- 24 hours
- 48 hours
- 168 hours

These features allow the model to identify how previous pollutant levels affect future AQI.

---

17. Rolling Features

Rolling statistical features were generated to capture short-term and long-term trends.

For AQI, the following statistics were calculated:

- Mean
- Standard deviation
- Minimum
- Maximum

The rolling windows included:

- 3 hours
- 6 hours
- 12 hours
- 24 hours
- 48 hours
- 72 hours
- 168 hours

Rolling statistics were also generated for major pollutants.

One of the most important features identified during model analysis was:

pm25_rolling_mean_24

This represents the average PM2.5 concentration over the previous 24 hours.

---

18. Change Rate Features

Change features were generated to capture the rate at which environmental variables were changing.

AQI changes were calculated over:

- 1 hour
- 3 hours
- 6 hours
- 12 hours
- 24 hours
- 48 hours
- 72 hours

Similar change features were generated for major pollutants.

These features help the model detect increasing or decreasing pollution trends.

---

19. Forecast Targets

The forecasting problem was formulated as a multi-output regression problem.

The final dataset contains 72 future AQI targets:

target_aqi_t_plus_1

target_aqi_t_plus_2

target_aqi_t_plus_3

...

target_aqi_t_plus_72

Each target represents the AQI at a future hourly interval.

For example:

- t+1 = one hour ahead

- t+6 = six hours ahead
- t+24 = one day ahead
- t+48 = two days ahead
- t+72 = three days ahead

The final feature dataset contained:

14,352 rows and 237 columns.

It was stored at:

data/features/karachi_features_v2.csv

---

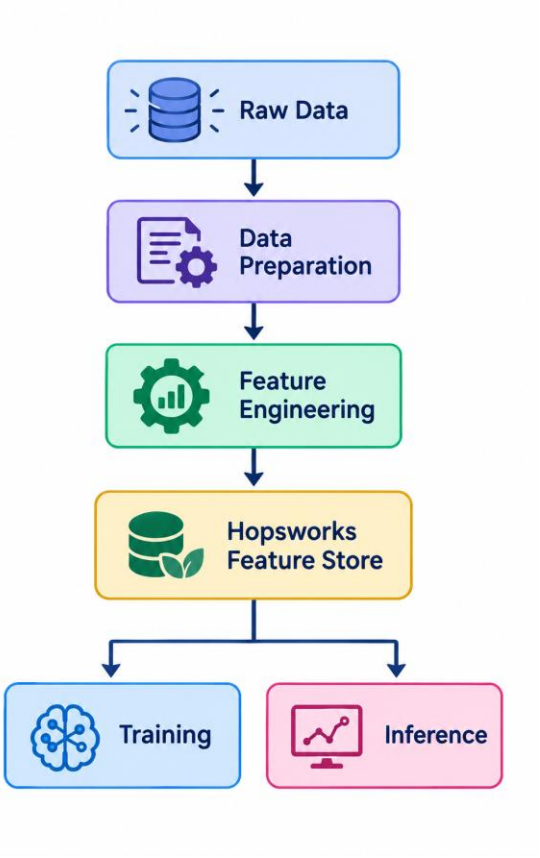
20. Hopsworks Feature Store Implementation

A major component of the project was the implementation of a **Feature Store using Hopsworks**.

The purpose of the Feature Store was to provide a structured and reusable location for machine learning features.

Instead of tightly coupling feature generation with model training, the engineered features can be stored and subsequently retrieved by the training pipeline.

The Hopsworks implementation provides the following workflow:



The feature engineering pipeline prepares the historical data and creates the required lag, rolling, temporal and target features.

The resulting features are then made available through Hopsworks for model training.

This architecture improves feature reuse and creates a cleaner separation between:

- Data processing
- Feature engineering
- Feature storage
- Model training
- Model inference

The Hopsworks Python client was used to establish the connection with the Hopsworks environment and interact with the project and Feature Store.

The implementation was structured so that the forecasting pipeline could retrieve the required features from the Feature Store rather than relying entirely on manually maintained feature files.

---

21. Machine Learning Models

Several forecasting approaches were evaluated:

1. Persistence baseline
2. Ridge Regression
3. Random Forest
4. XGBoost

The models were evaluated using the same time-series test methodology.

The purpose was to determine which approach could best model the relationship between historical environmental conditions and future AQI.

---

22. Persistence Baseline

The persistence model assumes that future AQI remains similar to the current AQI.

It was used as a simple baseline.

Horizon	MAE	RMSE	R^2
1 hour	0.426	1.073	0.991
6 hours	2.227	3.917	0.883
12 hours	3.862	5.876	0.738
24 hours	6.354	9.224	0.359
48 hours	9.402	13.116	-0.293
72 hours	11.067	15.503	-0.803

The baseline performed well for very short horizons but deteriorated as the forecast horizon increased.

---

23. Ridge Regression

Ridge Regression was used as a linear benchmark.

The initial evaluation produced:

- **MAE: 0.6126**
- **RMSE: 0.9117**

- **R^2: 0.9937**

Ridge provided a useful linear baseline.

However, AQI depends on complex and potentially non-linear interactions between pollutants, weather variables and temporal patterns.

Therefore, tree-based models were also evaluated.

---

24. Random Forest

Random Forest was trained using the engineered feature set.

The initial evaluation produced:

- **MAE: 0.3497**
- **RMSE: 0.6493**
- **R^2: 0.9968**

Random Forest performed better than Ridge Regression in the initial comparison.

Feature importance analysis was also performed using the Random Forest model to identify influential predictors.

---

25. XGBoost

XGBoost was selected for the final 72-hour forecasting system.

The model was trained using:

- **14,352 feature rows**
- **164 input features**
- **72 target variables**

The training dataset contained approximately 11,481 observations, while the test dataset contained approximately 2,871 observations.

The XGBoost model demonstrated particularly strong short-term forecasting performance.

Forecast Horizon	MAE	RMSE	R^2
1 hour	0.330	0.516	0.998
6 hours	1.526	2.250	0.962

12 hours	7.708	9.444	0.327
24 hours	7.323	9.653	0.300
48 hours	10.445	12.948	-0.243
72 hours	11.161	13.980	-0.446

The strongest result was obtained at the one-hour forecast horizon:

MAE = 0.330
RMSE = 0.516
R^2 = 0.998

At six hours, the model achieved:

MAE = 1.526
RMSE = 2.250
R^2 = 0.962

As the forecast horizon increased, prediction difficulty also increased.

The trained model artifact was stored as:

models/xgboost_72h.pkl

---

26. Model Comparison

The initial model comparison showed that tree-based models performed better than the linear model.

Model	MAE	RMSE	R^2
Ridge Regression	0.613	0.912	0.994
Random Forest	0.350	0.649	0.997

The persistence model was used as a reference point.

XGBoost was selected for the final 72-hour forecasting application because of its strong short-term performance and ability to model non-linear relationships.

---

27. Feature Importance

Feature importance analysis was performed to understand which variables had the greatest influence on the model.

The major features included:

Feature	Importance
pm25_rolling_mean_24	0.363
month_cos	0.081
day_of_month	0.074
pm25	0.048
pm10_rolling_mean_24	0.044
pressure	0.030
aqi_lag_72	0.025
month	0.025
day_of_week	0.018
aqi_rolling_mean_24	0.017

The most important feature was:

pm25_rolling_mean_24

with an importance of approximately **0.363**.

This indicates that the recent 24-hour PM2.5 trend plays an important role in predicting AQI.

The feature importance results were saved in:

models/feature_importance.csv

---

28. SHAP Explainability

Model accuracy alone is not sufficient for an environmental forecasting application.

Users should also have an understanding of **why** a model produces a particular prediction.

Therefore, SHAP was integrated into the project.

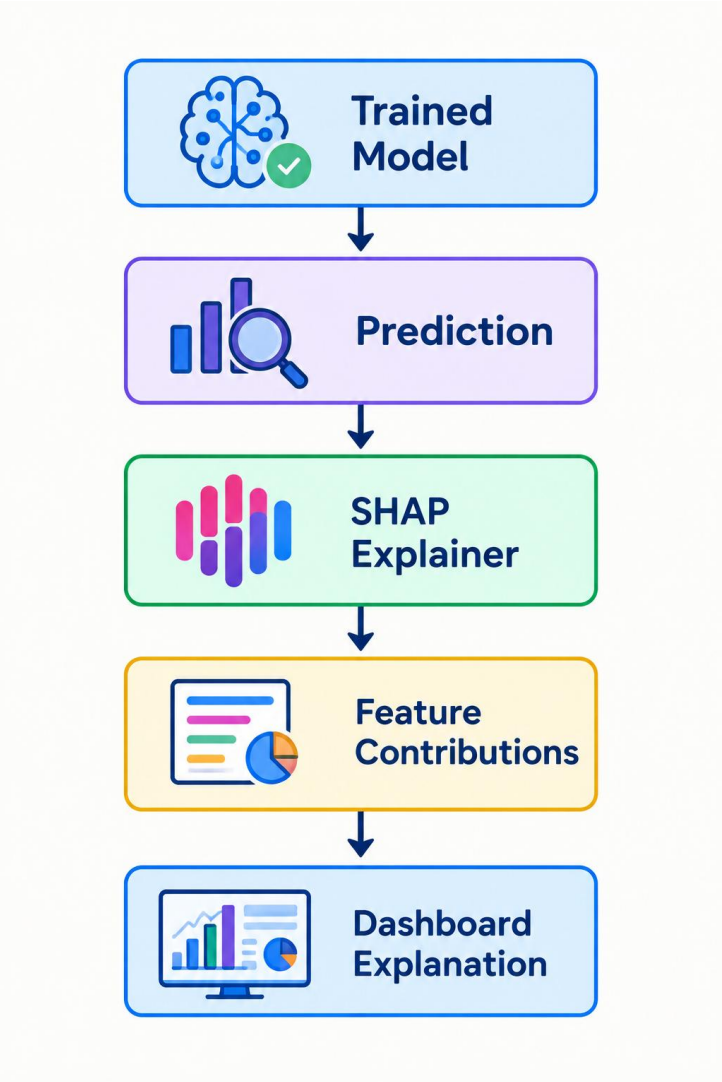
SHAP (SHapley Additive exPlanations) is a model explainability technique that estimates the contribution of individual features to a prediction.

The SHAP implementation was used to analyze the contribution of features such as:

- PM2.5
- PM10
- AQI history

- Rolling pollution statistics
- Weather conditions
- Time-related features

The explainability workflow follows:



SHAP analysis was used to generate feature-level explanations and overall feature importance.

The analysis produced files including:

models/shap_importance.csv

and:

models/shap_summary.png

The dashboard uses the explainability results to help users understand which features are influencing the AQI forecast.

---

29. Hopsworks Model and MLOps Integration

Hopsworks was incorporated into the project not only for feature management but also as part of the broader machine learning lifecycle.

The project structure maintains information about:

- Model name
- Model version
- Training information
- Feature version
- Forecast horizon
- Evaluation metrics
- Model artifact
- Production status

The model-management structure makes it possible to identify which model version is currently being used by the forecasting application.

The production model is:

Karachi AQI XGBoost 72H – Version 1.0.0

The model artifact is:

models/xgboost_72h.pkl

This creates a foundation for future model versions and makes the system easier to maintain.

---

30. Automated Data Pipeline

GitHub Actions was used to automate the data pipeline.

The hourly workflow performs tasks such as:

1. Checkout the repository.
2. Set up Python.
3. Install project dependencies.
4. Collect the latest AQI data.

5. Collect weather information.
6. Prepare the latest dataset.
7. Generate updated features.
8. Update the feature pipeline.

The workflow can also be triggered manually for testing.

---

31. Automated Model Training Pipeline

A separate daily workflow was developed for model training.

The daily pipeline performs:

1. Repository checkout.
2. Python environment setup.
3. Dependency installation.
4. Feature generation/retrieval.
5. XGBoost model training.
6. Model evaluation.
7. SHAP analysis.
8. Model artifact generation.
9. Model metadata updates.

This automation allows the model to be retrained periodically as new environmental observations become available.

---

32. Streamlit Dashboard

A Streamlit dashboard was developed as the primary user interface.

The dashboard converts the machine learning outputs into an accessible visual format.

The dashboard was designed around three major requirements:

1. Current Reading

The dashboard displays the current AQI reading so users can immediately understand the present air-quality condition.

2. Three-Day Forecast

The dashboard displays the predicted AQI for the next **72 hours**.

This provides a three-day view of expected air-quality conditions.

3. SHAP Explainability

The dashboard provides an explainability section that identifies the features influencing the AQI prediction.

This allows users to understand not only the predicted AQI but also the major factors contributing to it.

---

33. Dashboard — Current AQI Reading

The current AQI section provides the latest available air-quality measurement.

The dashboard can display:

- Current AQI value
- AQI category
- Latest timestamp
- Current pollutant information

The AQI category provides a simple interpretation of the numerical value.

This allows users to quickly determine whether current air quality is Good, Moderate or within a more concerning category.

---

34. Dashboard — 3-Day AQI Forecast

The main forecasting component displays the next **72 hours of predicted AQI**.

The forecast is generated by the XGBoost multi-output model.

The dashboard presents the forecast in a visual format, allowing users to identify:

- Increasing AQI trends
- Decreasing AQI trends
- Potential pollution peaks
- Changes over the three-day period

The 72-hour forecast corresponds to:

Day 1 → 0–24 hours

Day 2 → 24–48 hours

Day 3 → 48–72 hours

This provides users with both short-term and extended air-quality information.

---

35. Dashboard — SHAP Explainability

The SHAP section of the dashboard provides model transparency.

The purpose is to answer:

Why is the model predicting this AQI?

The explainability section can show the features that have the greatest influence on the prediction.

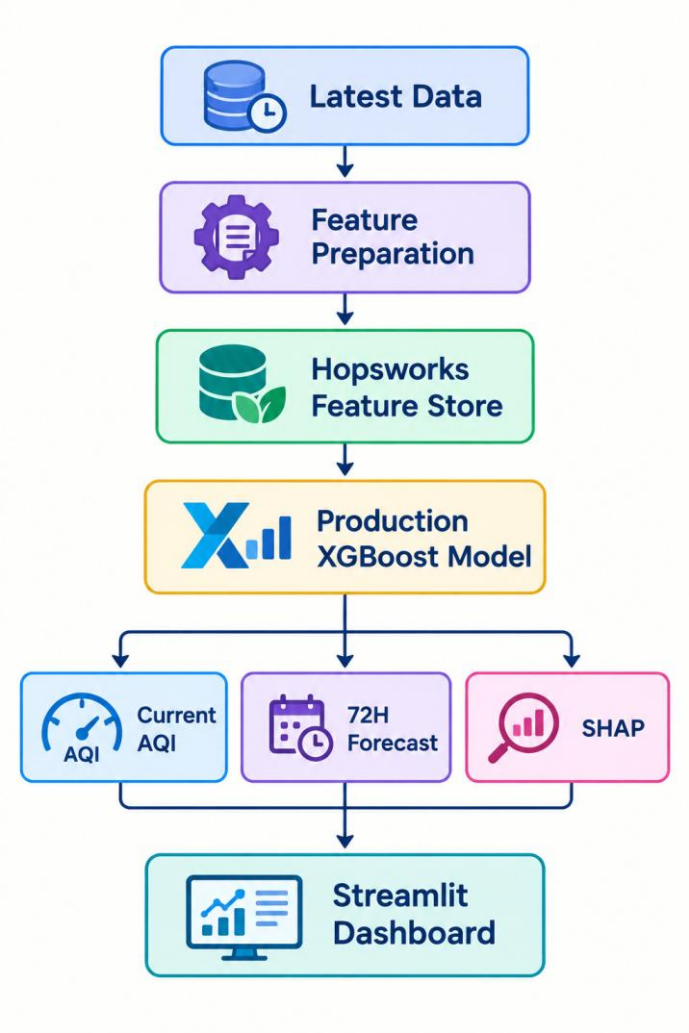
For example, features associated with recent PM2.5 levels, rolling pollution averages, historical AQI and temporal conditions can contribute to the forecast.

This makes the application more transparent than a dashboard that only displays a prediction value.

---

36. Dashboard Workflow

The dashboard follows the following workflow:



The interface therefore connects the machine learning infrastructure directly to the end user.

37. Deployment

The Streamlit application was deployed using **Streamlit Community Cloud**.

The deployment is connected to the project's GitHub repository.

The application uses:

- GitHub repository
- Main branch
- app.py
- requirements.txt
- Trained model

- Feature data/infrastructure
- SHAP explainability components

The application is designed to provide the current AQI, three-day forecast and explainability through a single web interface.

Dashboard URL:
https://pearls-aqi-predictor-hieth5onniz5tquubs8odr.streamlit.app/

GitHub Repository:
https://github.com/AreebaMandavia/pearls-aqi-predictor

---

38. Testing

The system was tested at multiple stages.

38.1 Data Testing

The historical datasets were checked for:

- Missing values
- Duplicate timestamps
- Correct record counts
- Correct data types
- Successful merging

38.2 Feature Testing

The feature dataset was checked for:

- Time features
- Lag features
- Rolling statistics
- Change features
- Pollutant features
- 72 future target columns

The final dataset contains target variables from:

t+1 through t+72.

38.3 Model Testing

The machine learning models were evaluated using:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2

Time-series evaluation was used to avoid randomly mixing future and historical observations.

38.4 Hopsworks Testing

The Hopsworks integration was tested for:

- Project connectivity
- Feature Store access
- Feature availability
- Feature retrieval
- Compatibility with the model-training workflow

38.5 SHAP Testing

The explainability pipeline was tested to ensure that model predictions could be analyzed and that feature contribution information could be generated for the dashboard.

Because the final forecasting model uses a multi-output structure, SHAP integration required additional handling so that predictions could be explained appropriately.

38.6 Automation Testing

Both GitHub Actions workflows were manually tested.

The main workflows were:

- Hourly AQI Pipeline
- Daily Model Training Pipeline

The workflows were configured for scheduled execution as well as manual execution.

38.7 Dashboard Testing

The Streamlit dashboard was tested for its primary components:

- Current AQI display
- 72-hour forecast
- Forecast visualization
- SHAP explainability
- Model information
- Error handling

---

39. Challenges Faced

Several technical challenges were encountered during development.

39.1 Historical API Data

Historical data collection required handling date ranges and API limitations.

The solution involved processing valid historical periods and combining the resulting datasets.

39.2 Time-Series Feature Engineering

Generating 72 future target variables required careful handling of timestamps.

Rows near the end of the dataset did not contain all 72 future AQI values and therefore had to be excluded from the final training dataset.

39.3 Long-Term Forecasting

The model performed strongly at shorter horizons but performance decreased for longer forecasts.

This is expected because atmospheric conditions can change significantly over a period of several days.

39.4 Hopsworks Integration

Integrating Hopsworks required configuring the Python environment, connecting to the Hopsworks project and ensuring that the feature pipeline could interact correctly with the Feature Store.

Additional dependency and environment configuration was required for the machine learning pipeline.

39.5 SHAP and Multi-Output Models

One of the major technical challenges was integrating SHAP with the final multi-output forecasting model.

The forecasting model produces multiple AQI predictions corresponding to different future horizons.

SHAP explainability therefore required special handling rather than simply applying a standard single-output TreeExplainer directly to the complete multi-output wrapper.

This was an important practical lesson in model explainability and compatibility between machine learning libraries.

39.6 Dashboard Integration

The dashboard had to connect several components:

- Current data
- Feature data
- Forecasting model
- 72-hour predictions
- SHAP explanations

This required careful handling of model inputs, feature names, prediction outputs and error conditions.

---

40. Limitations

Although the project provides an end-to-end forecasting system, several limitations remain.

40.1 Long-Term Forecast Accuracy

The model performs substantially better at short horizons than at 48- or 72-hour horizons.

This is a common challenge in environmental time-series forecasting.

40.2 Location Specificity

The current system is optimized for Karachi.

It is not directly designed for other cities.

Supporting other locations would require additional historical data and potentially separate models.

40.3 Environmental Variability

Unexpected weather events, pollution sources and atmospheric changes may cause actual AQI values to differ significantly from predicted values.

40.4 Feature Store Dependency

The cloud-based feature-management architecture introduces dependency on the availability and configuration of the Hopsworks environment.

40.5 Explainability of Multi-Step Forecasts

Because the model generates predictions for multiple future horizons, explaining every forecast horizon independently can be more complex than explaining a single prediction.

---

41. Future Improvements

Several improvements can be made in future versions.

41.1 Advanced Forecasting Models

Future work could evaluate:

- LSTM
- GRU
- Temporal Convolutional Networks
- Transformers
- Temporal Fusion Transformers
- SARIMA
- Prophet

These models could be compared against XGBoost.

41.2 Improved Multi-Step Forecasting

A dedicated sequence forecasting model could be developed to model dependencies between consecutive forecast horizons.

This could potentially improve the consistency of the 72-hour forecast.

41.3 More Historical Data

Additional years of historical data could improve the model's understanding of seasonal patterns.

41.4 Multi-City Forecasting

The system could be extended to support multiple cities.

Users could select a city from the dashboard and receive location-specific AQI predictions.

41.5 Real-Time Alerts

The dashboard could be extended with:

- Email alerts
- SMS alerts
- Push notifications
- High-AQI warnings

41.6 Improved Hopsworks Integration

The Feature Store could be expanded with additional feature groups and more systematic feature versioning.

Feature reuse could also be extended to multiple forecasting models and multiple cities.

41.7 Advanced Model Management

A more advanced model-management system could be introduced for:

- Experiment tracking
- Model versioning
- Model lineage
- Model promotion
- Automated model comparison
- Model monitoring

---

42. Final Results

The Pearls AQI Predictor project successfully developed an end-to-end environmental machine learning system.

The major results include:

- **14,592 historical hourly AQI records**
- **14,592 historical weather records**
- **14,592 × 14 prepared dataset**
- **14,352 × 237 final feature dataset**
- **164 input features**
- **72 future AQI targets**
- Persistence baseline evaluated
- Ridge Regression evaluated
- Random Forest evaluated
- XGBoost evaluated
- XGBoost selected for 72-hour forecasting
- Hopsworks Feature Store implemented
- Model management implemented
- SHAP explainability implemented
- GitHub Actions automation implemented
- Streamlit dashboard developed
- Current AQI display implemented
- Three-day / 72-hour forecast implemented
- SHAP explainability integrated into the dashboard
- Online deployment configured

The strongest XGBoost result was achieved at the one-hour horizon:

MAE = 0.330
RMSE = 0.516
R^2 = 0.998

At the six-hour horizon:

MAE = 1.526
RMSE = 2.250
R^2 = 0.962

The results demonstrate that the system provides particularly strong short-term AQI predictions, while longer forecasting horizons remain more challenging.

---

43. Conclusion

The **Pearls AQI Predictor** project demonstrates how machine learning, data engineering, cloud infrastructure, explainable AI and web development can be combined to create a complete environmental forecasting system.

The project began with raw air-quality and weather observations and evolved into an automated end-to-end machine learning pipeline.

The system performs:

Data Collection → Data Preparation → Feature Engineering → Hopsworks Feature Store → Model Training → Model Evaluation → SHAP Explainability → Streamlit Dashboard

Several machine learning models were evaluated, including a persistence baseline, Ridge Regression, Random Forest and XGBoost.

XGBoost was selected as the final forecasting model because it produced strong short-term results and was capable of learning complex non-linear relationships between historical AQI, pollutant and weather features.

The implementation of **Hopsworks Feature Store** provided a structured approach to feature management and reuse. This helped separate feature engineering from model training and created a foundation for a more scalable MLOps architecture.

The integration of **SHAP explainability** added transparency to the forecasting system. Instead of only presenting an AQI prediction, the application can provide information about the features influencing that prediction.

The **Streamlit dashboard** serves as the final user interface and focuses on three important functions: displaying the current AQI, providing a three-day forecast and explaining predictions using SHAP.

GitHub Actions further automated the recurring data and model workflows.

The project also demonstrated an important characteristic of environmental forecasting: short-term predictions can achieve high accuracy, while longer-term predictions become increasingly difficult because atmospheric conditions are dynamic.

Overall, the internship provided practical experience in the complete machine learning lifecycle, including data collection, feature engineering, Feature Store implementation, model training, model evaluation, explainable AI, automation and deployment.

The resulting system provides a strong foundation for future development involving advanced forecasting architectures, multi-city prediction, real-time alerts and more sophisticated MLOps capabilities.

---

44. Appendix

A. Important Project Files

File	Purpose
collect_data.py	Collects current AQI and weather data
backfill.py	Collects historical AQI data
backfill_weather.py	Collects historical weather data
prepare_data.py	Combines AQI and weather data
feature_engineering.py	Generates machine learning features
feature_store.py	Handles Feature Store operations
train_models.py	Trains baseline machine learning models
train_72hr.py	Trains 72-hour forecasting model
train_xgboost_72hr.py	Trains XGBoost 72-hour model
baseline_forecast.py	Evaluates persistence baseline
feature_importance.py	Generates feature importance
explain_model.py	Generates SHAP explanations
model_registry.py	Handles model metadata and registration
app.py	Streamlit dashboard

B. Main Output Files

data/processed/karachi_complete.csv

data/processed/karachi_eda.csv

data/features/karachi_features_v2.csv

models/xgboost_72h.pkl

models/model_registry.json

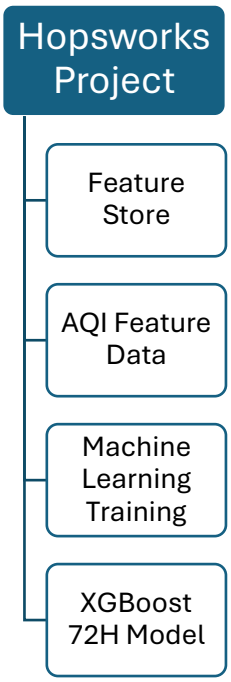
models/feature_importance.csv

models/shap_importance.csv

models/shap_summary.png

C. Hopsworks Components

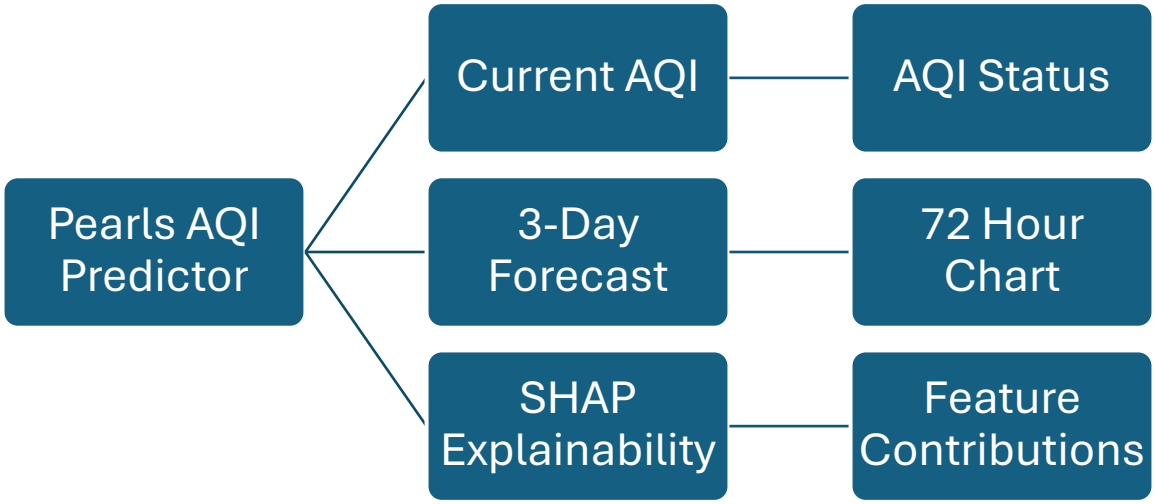
The Hopsworks implementation includes:



The Feature Store provides centralized access to engineered machine learning features and separates feature management from the model-training process.

D. Dashboard Components

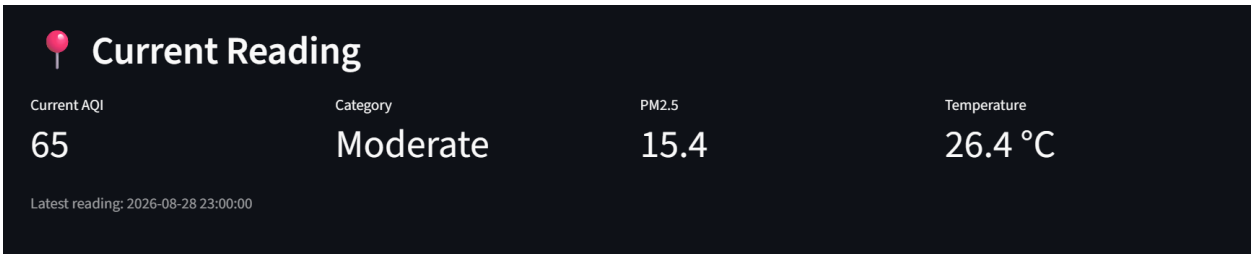
The final dashboard is organized around:



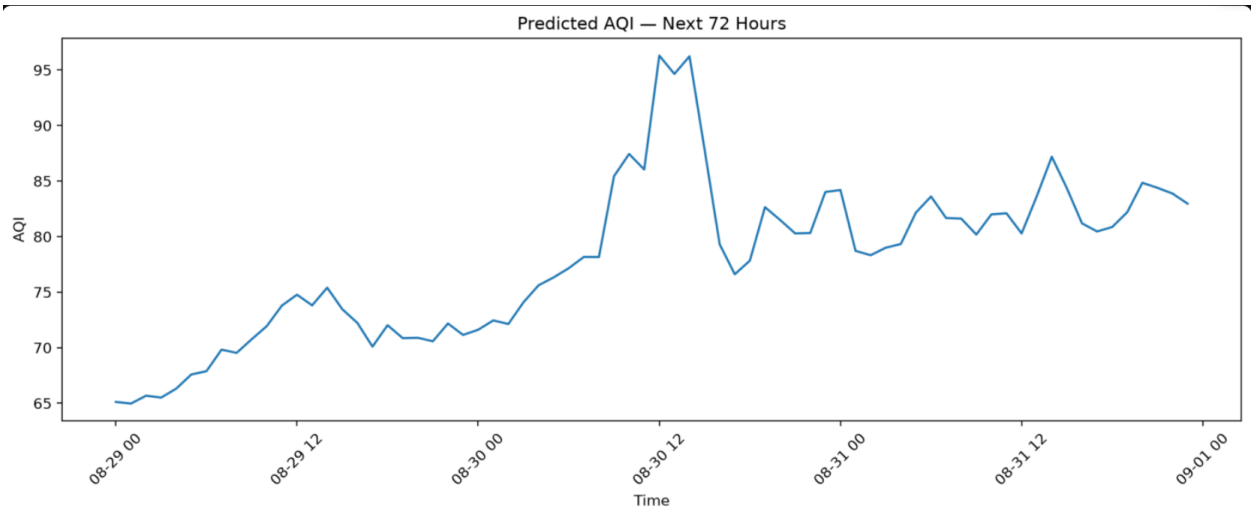
The dashboard therefore provides both **prediction and interpretation**.

E. Screenshots

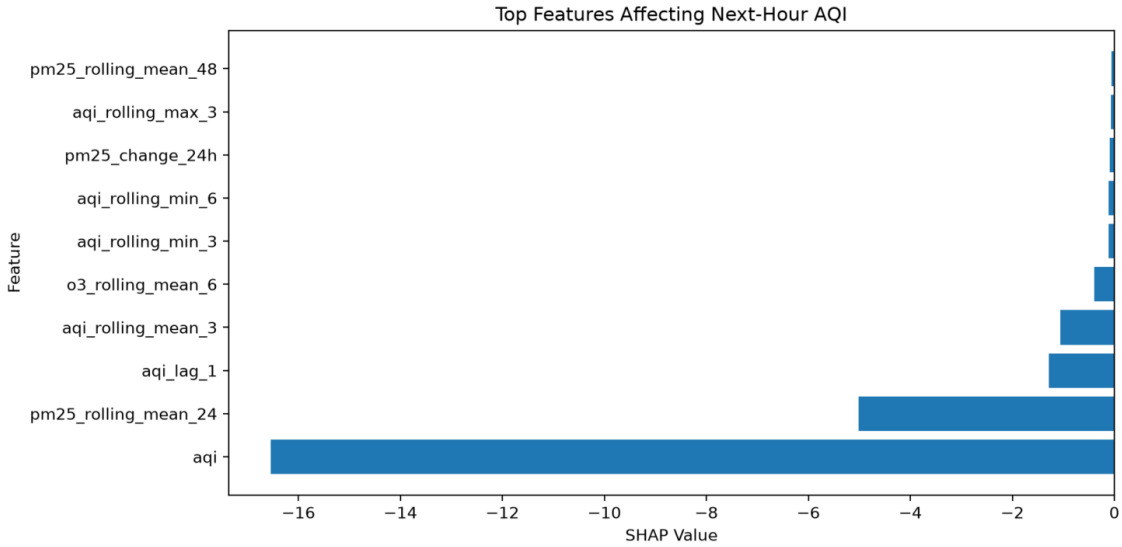
1. Current AQI reading



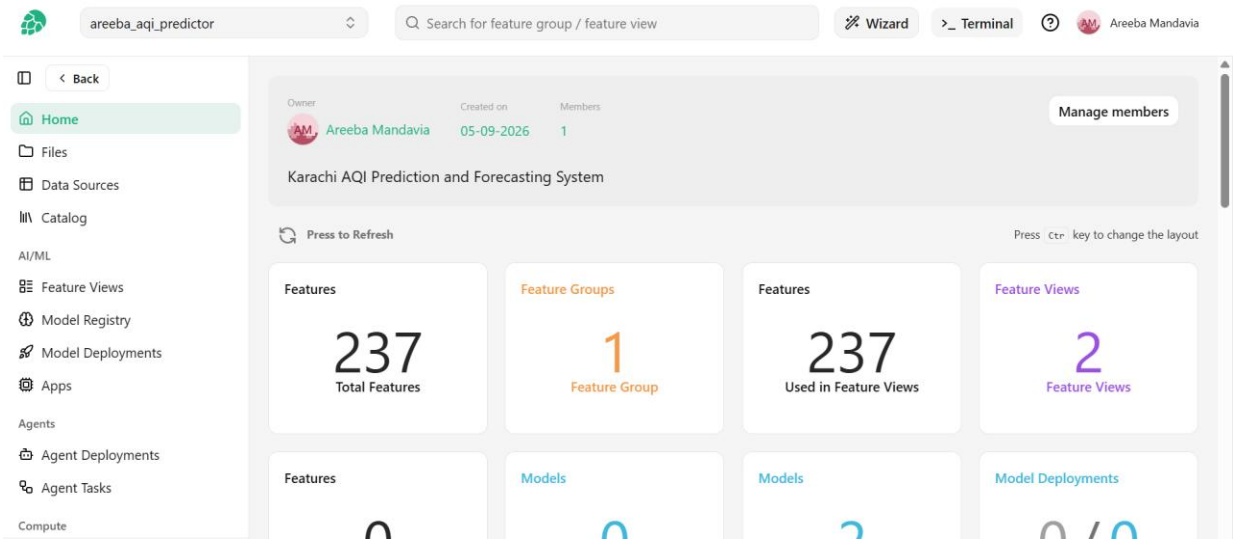
2. 72-hour forecast chart



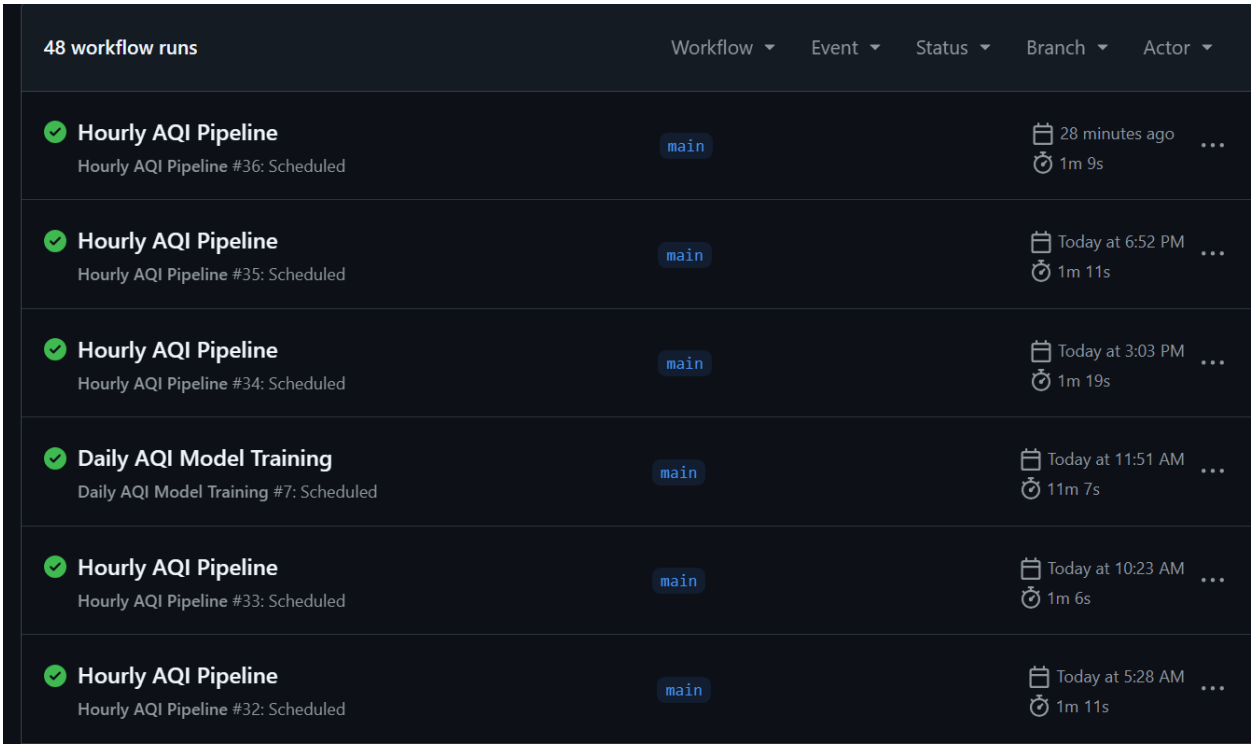
3. SHAP explainability



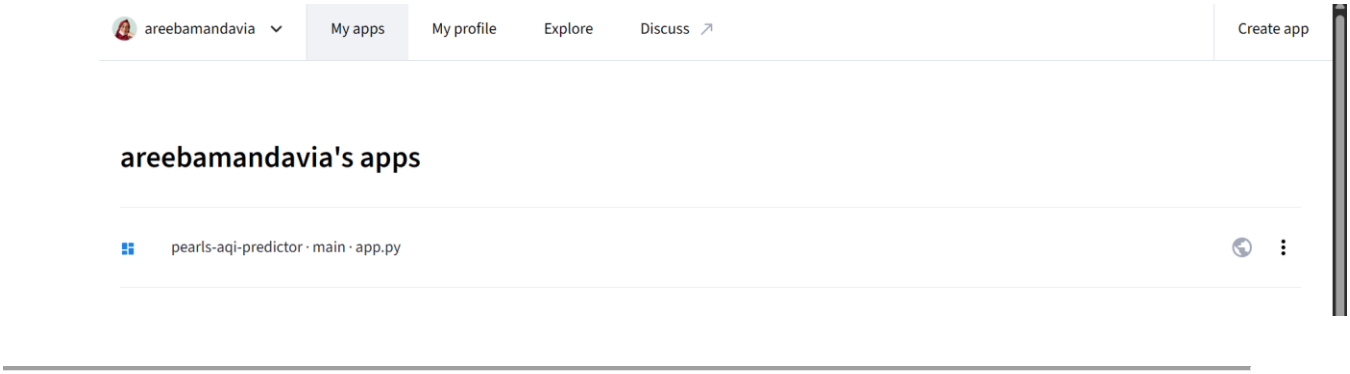
4. Hopsworks project



5. Successful GitHub Actions workflow



6. Streamlit deployed application


areebamandavia
My apps
My profile
Explore
Discuss
Create app
areebamandavia's apps
pearls-aqi-predictor · main · app.py